



FareLens

Amadeus NDC fare rules and amenities, explained.

Self-hosted API + admin UI that turns the fare rules and fare-family amenities from Amadeus NDC responses into traveller-friendly summaries, amenity cards, friendly fare names and answers to traveller questions, using the LLM provider you choose.

Product & Technical Guide · Version 1.0.0

September 2026

Built by TravelSwitch for the travel-tech community · travelswitch.com
Open source under the MIT License

Contents

1. Introduction	3
1.1 Why FareLens	3
1.2 What you get	3
1.3 The admin interface at a glance	4
2. Quick start.....	5
2.1 Run it.....	5
2.2 Choose an LLM provider	5
2.3 Create an API key	5
2.4 Call the API.....	6
2.5 Try it in the Playground	6
2.6 Watch usage	8
3. API reference	9
3.1 Authentication and errors	9
3.2 POST /api/v1/fare-rules/summary	9
3.3 POST /api/v1/fare-rules/chat/stream	10
3.4 Other endpoints.....	10
3.5 POST /api/v1/amenities/summary	10
3.6 POST /api/v1/amenities/fare-names.....	12
3.7 Interactive documentation (Swagger UI).....	13
4. Configuration	16
4.1 Environment variables.....	16
4.2 LLM providers	16
4.3 Using your own Postgres / Redis	17
4.4 Prompts	17
5. Architecture	19
5.1 Request flow	19
5.2 Code layout.....	19
5.3 Technology.....	19
6. Deployment	20
6.1 Production checklist	20
6.2 Reverse proxy	20
6.3 Scaling, persistence and upgrades.....	20
6.4 What FareLens stores	21
7. Contributing, credits and license	22

1. Introduction

1.1 Why FareLens

Amadeus NDC fare rules are long, uppercase, telegraphic and full of ATPCO jargon ([NON-REF](#), [RFND](#), [CHG](#) [PEN](#), [NOSHOW](#)...). Travellers just want to know: **what does it cost me to cancel or change?** FareLens answers that in plain language, in the traveller's own language, formatted for desktop or mobile — and caches the result so repeated fare rules cost zero tokens.

FareLens was built by TravelSwitch, where it powers fare-rule explanations in production booking flows, and is shared with the travel-tech community so any airline, OTA or travel-tech team can run it on their own stack.

The problem this solves for Amadeus NDC integrators

If you consume airline content through Amadeus NDC, you will have hit this:

- **Fare-family names, amenity lists and fare-rule text are passed through exactly as the airline supplies them.** There is no Amadeus-generated short version the way there is for EDIFACT content.
- **There is no parameter to control the length**, and no standardisation across carriers.
- **The text is often long and inconsistent**, with detailed conditions or URLs embedded, which makes it unusable in a B2C booking flow.
- **This is by design.** When raised with Amadeus, the position is that the behaviour is compliant with the NDC standard, so the integrator has to solve it.

FareLens is that solution. Feed it whatever the airline returned and get back a consistent, short, structured summary, plus a Q&A channel for the follow-up questions travellers actually ask.

FareLens is that solution for both halves of the fare-family payload: the rules and the amenity list, plus friendly fare-family names.

1.2 What you get

Capability	Description
Summary API	POST /api/v1/fare-rules/summary → Markdown "mini rules": a table on desktop, compact sections on mobile.
Chat API	POST /api/v1/fare-rules/chat/stream → Server-Sent Events Q&A over one or many itinerary segments; segment- and date-aware.
Amenities API	POST /api/v1/amenities/summary → raw fare-family benefit texts in; de-duplicated, classified, translated items out. POST /api/v1/amenities/fare-names → ECOLITE becomes "Economy Lite".
Bring your own model	OpenAI · Azure AI (Azure OpenAI) · Anthropic Claude · Google Gemini · Groq · AWS Bedrock. Pick one in the UI, test it, switch any time.
Multilingual	Arabic, Urdu, French, German, Hindi, Chinese and more — the response language is a request parameter.
Caching	Content-digest cache: Redis → Postgres → model. Identical rules never hit the LLM twice.
Admin UI	Provider setup, API keys, usage dashboard, datastore management, playground, account.
One command	docker compose up -d brings up the API, Postgres and Redis with zero configuration.

1.3 The admin interface at a glance

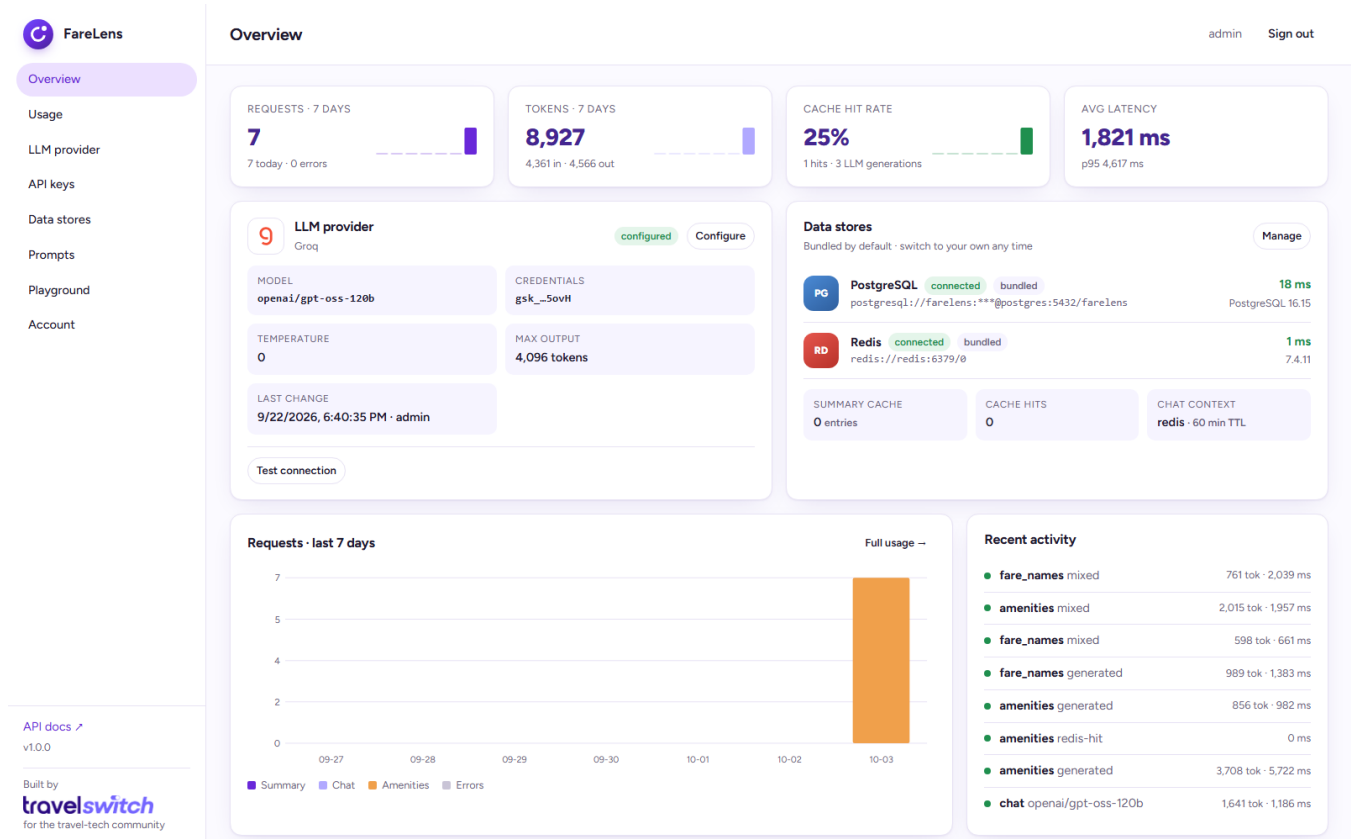


Figure 1 – Overview dashboard: usage tiles, provider status, live datastore checks, recent activity

2. Quick start

Requirements: Docker with Compose v2. Nothing else — no Python, no database to install.

2.1 Run it

```
git clone https://github.com/travelswitch/farelens.git
cd farelens
docker compose up -d
```

Open **http://localhost:8000** and sign in with the default credentials **admin** / **admin**. The UI asks you to change the password on first login.

2.2 Choose an LLM provider

Pick a provider, paste your key, click **Test connection**, then **Save & activate**. Credentials are encrypted at rest. Any model id the provider accepts can be typed in; the suggested list is only a starting point.

The screenshot displays the FareLens web interface for configuring an LLM provider. On the left, a sidebar contains navigation links: Overview, Usage, LLM provider (highlighted), API keys, Data stores, Prompts, Playground, and Account. The main panel is titled 'LLM provider' and features a 'Choose an inference provider' section with buttons for OpenAI, Azure AI (Azure OpenAI), Anthropic (Claude), Google (Gemini), Groq (marked 'active'), and AWS Bedrock. Below this, the 'Active configuration' for Groq is detailed, showing the model 'openai/gpt-oss-120b', API key 'gsk_5ovHt', temperature '0', and max output tokens '4096'. A 'Good to know' section lists important notes about secrets and caching. At the bottom, there are links to 'API docs v1.0.0' and a note about being built by travelswitch for the travel-tech community.

Figure 2 – Provider picker with connection test

2.3 Create an API key

API keys authenticate the public endpoints. The full key is shown once; only a hash is stored.

API keys

Create a key

Name, e.g. booking-web [Create API key](#)

Send it as X-API-Key: f1_... (or Authorization: Bearer f1_...). The full key is shown once.

Keys

NAME	PREFIX	CREATED	LAST USED	REQUESTS	STATUS
demo	f1_Xg2R4ez8...	9/22/2026, 6:40:47 PM admin	10/3/2026, 9:41:35 AM	31	active Revoke

[API docs v1.0.0](#)

Built by **travelswitch** for the travel-tech community

Figure 3 – API key management

2.4 Call the API

```
curl -X POST http://localhost:8000/api/v1/fare-rules/summary \
-H "X-API-Key: f1_..." \
-H "Content-Type: application/json" \
-d '{"fare_rules_text": "CANCELLATIONS BEFORE DEPARTURE CHARGE SAR 150 FOR CANCEL/REFUND ...",
"lang": "en"}'
```

Interactive OpenAPI documentation is served at **<http://localhost:8000/docs>** (see Section 3.5).

2.5 Try it in the Playground

Both endpoints can be exercised from the UI without an API key (the admin session is used). Load a sample, choose a language and layout, and see the result rendered.



Overview
Usage
LLM provider
API keys
Data stores
Prompts
Playground
Account

API docs
v1.0.0

Built by
travelswitch
for the travel-tech community

Playground

admin Sign out

Summary Chat Amenities Fare names

Runs with your admin session - recorded in Usage

Fare rules

Paste raw airline text — HTML and uppercase are fine

Load sample...

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 150 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.
NO-SHOW
TICKET IS NON-REFUNDABLE.

CHANGES
BEFORE DEPARTURE
CHARGE SAR 100 FOR REISSUE/REVALIDATION. FARE DIFFERENCE APPLIES.
CHANGES PERMITTED UP TO 4 HOURS BEFORE DEPARTURE.
AFTER DEPARTURE
CHARGE SAR 200 FOR REISSUE. FARE DIFFERENCE APPLIES.
NO-SHOW
CHARGE SAR 250 FOR REISSUE. FARE DIFFERENCE APPLIES.

Language English Layout Desktop Mobile Bypass cache 589 chars

Summarise →

Result

generated by model: groq - openai/gpt-oss-120b - 2,715 ms

Copy Markdown Raw

Section**	Condition / Timing	Penalty	Applicability	**Notes
Cancellation / Refund	Before departure	SAR 150	All	Child/infant discounts apply; waived for death of passenger or family member.
Cancellation / Refund	After departure	Non-refundable	All	Refund of unused taxes permitted.
Cancellation / Refund	No-show	Non-refundable	All	—
Change / Reissue	Before departure (up to 4 h before departure)	SAR 100	All	Fare difference applies; changes permitted up to 4 h before departure.
Change / Reissue	After departure	SAR 200	All	Fare difference applies.
Change / Reissue	No-show	SAR 250	All	Fare difference applies.
Other	Ticket validity	1 year from date of issue	All	—

Key points

- Cancellation before departure costs **SAR 150** (discounts for child/infant; waiver for death).
- After departure the ticket is **non-refundable**; only unused taxes may be returned.
- Changing a ticket before departure (≤ 4 h) costs **SAR 100**; fees rise to **SAR 200** after departure and **SAR 250** for no-shows.

Figure 4 – Playground: fare-rules summary with cache badge, model and latency



Overview
Usage
LLM provider
API keys
Data stores
Prompts
Playground
Account

API docs
v1.0.0

Built by
travelswitch
for the travel-tech community

Playground

admin Sign out

Summary Chat Amenities Fare names

Runs with your admin session - recorded in Usage

Itinerary

One segment per flight, each with its own fare rules

+ Segment

DEL → DXB 10-Oct-2026

Fare rules for this segment - 589 chars

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 150 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.

DXB → DEL 17-Oct-2026

Fare rules for this segment - 589 chars

CANCELLATIONS
BEFORE DEPARTURE
CHARGE SAR 300 FOR CANCEL/REFUND.
CHILD/INFANT DISCOUNTS APPLY.
WAIVED FOR DEATH OF PASSENGER OR FAMILY MEMBER.
AFTER DEPARTURE
TICKET IS NON-REFUNDABLE.
REFUND OF UNUSED TAXES PERMITTED.

Language English Layout Desktop Mobile

Conversation

Conversation c5615942 - openai/gpt-oss-120b - 1,926 tokens - 975 ms

Reset

What does it cost to cancel the outbound flight? You



Outbound flight (DEL → DXB) – Cancellation cost

Segment	Timing	Penalty
DEL-DXB (outbound)	Before departure	SAR 150 (adult) – child/infant discounts apply; waived for death of passenger or family member
DEL-DXB (outbound)	After departure / no-show	Non-refundable (fare), only unused taxes may be refunded

So, if you cancel before the 10 Oct 2026 departure, the fee is **SAR 150** (plus any applicable child/infant discount). Not clearly specified in the fare rules for any other conditions (e.g., exact discount rates).

e.g. Can I cancel the return flight on 2026-10-12?

Send

Figure 5 – Playground: multi-segment chat with streaming answers

2.6 Watch usage

Every request is recorded: tokens in/out, latency, cache status, language and API key. The Usage page aggregates per day, per feature, per model and per key.

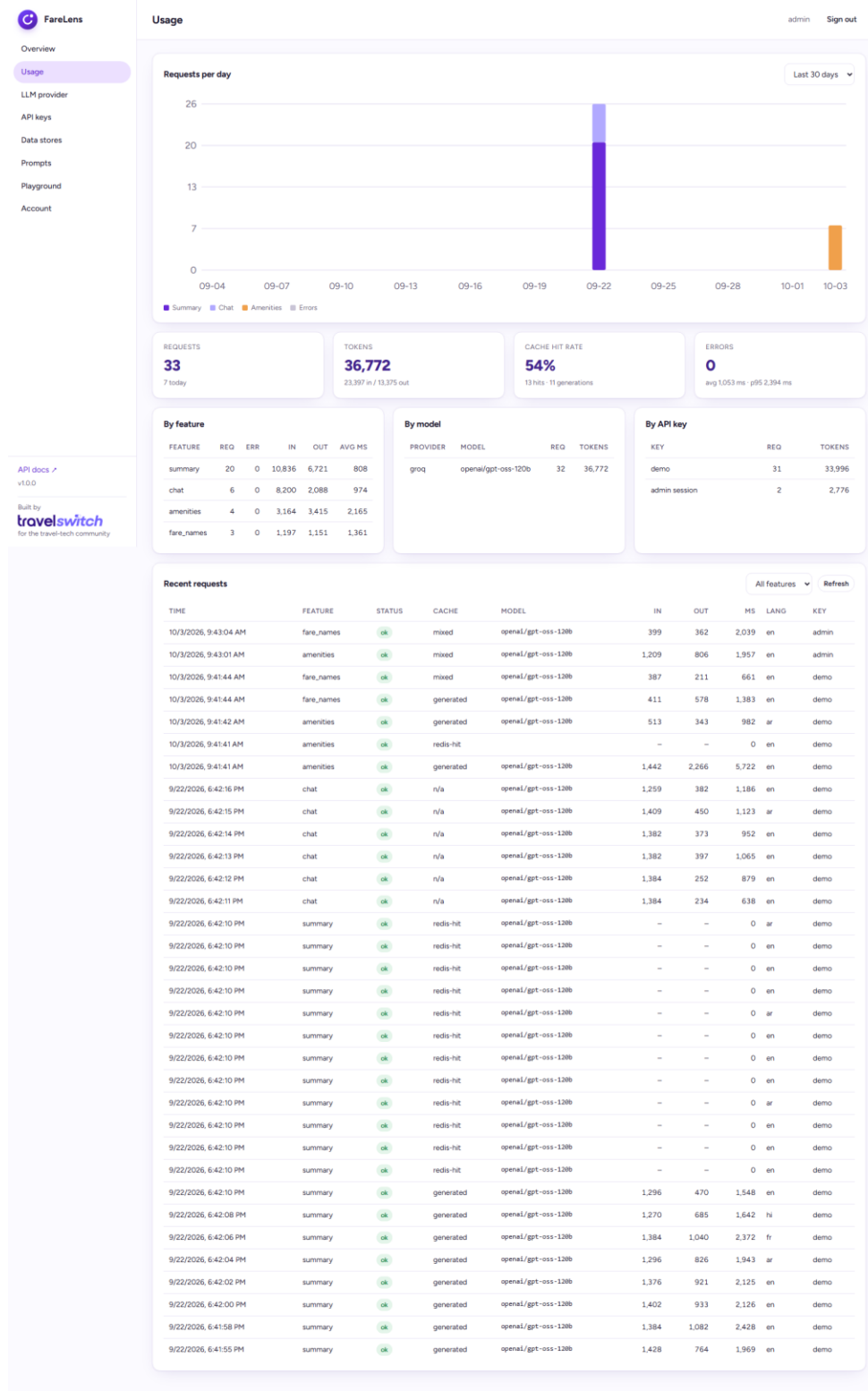


Figure 6 – Usage analytics

3. API reference

3.1 Authentication and errors

All public endpoints require an API key in the `X-API-Key` header (or `Authorization: Bearer fl_...`). Requests are rate-limited per key (default 120 per 60 s). Every response carries an `X-Request-Id` header; errors share one JSON envelope:

```
{"request_id": "...", "error": {"code": "LLM_NOT_CONFIGURED", "message": "...", "http_status": 503}}
```

HTTP	Code	Meaning
401	UNAUTHORIZED / INVALID_API_KEY	Missing or revoked key.
413	PAYLOAD_TOO_LARGE	Fare-rules text exceeds <code>MAX_FARE_RULES_CHARS</code> .
422	VALIDATION_ERROR / CONVERSATION_EXPIRED / SEGMENTS_REQUIRED	Bad request shape, or the chat context is gone — resend segments.
429	TOO_MANY_REQUESTS	Rate limit; see <code>Retry-After</code> .
502 / 503 / 504	LLM_*	Upstream provider problem: auth, model not found, rate limited, timeout, or no provider configured.

3.2 POST /api/v1/fare-rules/summary

Field	Type	Notes
<code>fare_rules_text</code>	string	Raw fare rules; HTML is stripped. Required.
<code>lang</code>	string	Response language code, default <code>en</code> .
<code>is_mobile_view</code>	bool	Compact sections instead of tables.
<code>bypass_cache</code>	bool	Force regeneration.

Response:

```
{
  "summary_markdown": "| Section | Condition / Timing | Penalty | Applicability | Notes |\n...",
  "cache": "generated",           // redis-hit | postgres-hit | generated (also in X-Cache
header)
  "provider": "groq",
  "model": "openai/gpt-oss-120b",
  "lang": "en",
  "is_mobile_view": false,
  "latency_ms": 1840
}
```

3.3 POST /api/v1/fare-rules/chat/stream

Start a conversation by sending the itinerary's segments. Follow-ups only need the `convo_id`, the new `user_message` and the client-side `history`. Segments are retained for `CONVERSATION_TTL_SECONDS` (default 1 hour).

```
{
  "segments": [
    {"source_airport": "DEL", "destination_airport": "DXB", "departure_date": "2026-11-06",
    "fare_rules_text": "..."},
    {"source_airport": "DXB", "destination_airport": "DEL", "departure_date": "2026-11-12",
    "fare_rules_text": "..."}
  ],
  "user_message": "Can I change the return flight?",
  "history": [],
  "lang": "en",
  "is_mobile_view": false
}
```

The response is `text/event-stream`:

```
event: start    data: {"convo_id": "...", "segments": [...]}
event: token    data: {"content": "Yes – "}
event: token    data: {"content": "changes on the return ..."}
event: end      data: {"convo_id": "...", "usage": {"prompt_tokens": 1450, "completion_tokens": 92},
"latency_ms": 2100}
event: error    data: {"error": {"code": "...", "message": "..."}}
```

Follow-up message:

```
{"convo_id": "...", "user_message": "And if I no-show?", "history": [{"role": "user", "content":
"..."}, {"role": "assistant", "content": "..."}]}
```

3.4 Other endpoints

Endpoint	Purpose
POST /api/v1/fare-rules/chat	Same request as the stream endpoint; single JSON response.
DELETE /api/v1/fare-rules/chat/{convo_id}	Forget a conversation's stored segments early.
GET /health	Liveness probe (reports Postgres/Redis connectivity).
GET /health/ready	Readiness probe (503 until Postgres is reachable; reports whether an LLM is configured).

3.5 POST /api/v1/amenities/summary

The other half of the fare-family payload is the benefit list: baggage, seat, lounge, meal, miles and so on. Airlines send it as repetitive, unstandardised text. This endpoint returns a short, de-duplicated, classified list in the traveller's language. The body can be an object or a bare JSON array of strings.

Field	Type	Notes
amenities	string[]	Raw benefit texts, one per line item (max 200).
lang	string	Output language code, default <code>en</code> .
summarize	bool	<code>true</code> (default): merge duplicates, classify, shorten. <code>false</code> : literal translation, one output per input.

```
{
  "amenities": [
    {"type": "CabinBaggage", "description": "1 cabin bag up to 7 kg", "details": "max 56x45x25 cm", "included": true, "is_chargeable": false, "ref_url": ""},
    {"type": "Lounge", "description": "Lounge access not included", "details": "", "included": false, "is_chargeable": false, "ref_url": ""},
    {"type": "Seat", "description": "Seat selection for a fee", "details": "", "included": false, "is_chargeable": true, "ref_url": "https://airline.example/seats"}
  ],
  "cache": "mixed", "lang": "en", "summarize": true, "provider": "groq", "model": "openai/gpt-oss-120b", "latency_ms": 1420
}
```

- Types: Baggage, CabinBaggage, Seat, Meal, Lounge, Priority, Wifi, Entertainment, Miles, Upgrade, Refund, Change, GroundServices, Insurance, Warning, Other.
- `included` says whether the benefit is provided (true), explicitly not included (false) or unclear (null); `is_chargeable` flags paid add-ons; URLs are moved to `ref_url`.
- Caching is per item and language: a batch of 20 where 18 were seen before sends only 2 texts to the model (`cache`: redis-hit, postgres-hit, generated or mixed).

The screenshot shows the FareLens Playground interface. On the left is a sidebar with navigation links: Overview, Usage, LLM provider, API keys, Data stores, Prompts, **Playground**, and Account. The main area has tabs for Summary, Chat, **Amenities**, and Fare names. Below the tabs, there's a section for 'Fare-family amenities' with a text area containing a list of amenities and a 'Process' button. To the right, a 'Result' section shows a grid of categorized amenities. Each item in the grid is labeled with its category, status (included/not included), and a brief description.

Category	Status	Description
CabinBaggage	included	1 cabin bag up to 7 kg
Baggage	included	1 checked bag up to 23 kg
Lounge	not included	Lounge access not included
Seat	not included	Seat selection for a fee
Miles	included	Earn 50% award miles
Meal	included	Complimentary snack and beverage
Change	not included	Changes permitted with fee
Priority	not included	Priority boarding not included

Figure 7 – Playground: amenities de-duplicated and classified

3.6 POST /api/v1/amenities/fare-names

Converts technical fare-family codes into short, friendly labels in the requested language. Results are cached per code and language.

```
{
  "names": ["ECOLITE", "BUSIFLEX", "PREMSAVR"],
  "lang": "en"
}
→ {
  "names": {
    "ECOLITE": "Economy Lite",
    "BUSIFLEX": "Business Flex",
    "PREMSAVR": "Premium Economy Saver"
  },
  "cache": "generated",
  ...
}
```

The screenshot shows the FareLens Playground interface. On the left is a sidebar with navigation links: Overview, Usage, LLM provider, API keys, Data stores, Prompts, Playground (highlighted), and Account. The main area is titled 'Playground' and has tabs for Summary, Chat, Amenities, and Fare names (selected). A status bar at the top right indicates 'admin' and 'Sign out', and a note says 'Runs with your admin session - recorded in Usage'. The 'Fare names' tab contains a text input field with the codes 'ECOLITE, ECOSHART, ECOFLEX, BUSIFLEX, PRENSAVR, FLEXPLUS, ECOXQ7, BIZPROMO'. Below the input is a 'Language' dropdown set to 'English' and a 'Convert →' button. To the right, a 'Friendly labels' panel displays a table mapping the codes to their friendly names. A cache indicator at the top of this panel shows 'cache - redis' and 'cache - 0 ms'.

CODE	LABEL
ECOLITE	Economy Lite
ECOSHART	Economy Standard
ECOFLEX	Economy Flex
BUSIFLEX	Business Flex
PRENSAVR	Premium Economy Saver
FLEXPLUS	Standard
ECOXQ7	Economy
BIZPROMO	Business

API docs [v1.0.0](#)

Built by **travelswitch** for the travel-tech community

Figure 8 – Playground: fare-family codes to friendly labels

3.7 Interactive documentation (Swagger UI)

The OpenAPI 3.1 specification is served at `/openapi.json` and rendered at `/docs` (Swagger UI) and `/redoc`. Every request and response schema can be inspected and tried live.

FareLens 1.0.0 OAS 3.1

[/openapi.json](#)

FareLens turns raw airline fare rules into traveller-friendly summaries, answers questions about them, and cleans up fare-family amenity lists and fare-family names, using the LLM provider you configure (OpenAI, Azure OpenAI, Anthropic, Google Gemini, Groq or AWS Bedrock).

Authenticate public endpoints with an API key created in the admin UI:

`X-API-Key: fl_...`

health

GET /health Liveness probe

GET /health/ready Readiness probe

fare-rules

POST /api/v1/fare-rules/summary Summarise fare rules into traveller-friendly Markdown

POST /api/v1/fare-rules/chat/stream Ask a question about the fare rules (Server-Sent Events)

POST /api/v1/fare-rules/chat Ask a question about the fare rules (single JSON response)

DELETE /api/v1/fare-rules/chat/{convo_id} Forget a conversation's stored segments

amenities

POST /api/v1/amenities/summary De-duplicate, classify and translate fare-family amenities

POST /api/v1/amenities/fare-names Convert fare-family codes to friendly labels

Schemas

AmenitiesResponse > Expand all object

AmenityItem > Expand all object

ChatMessage > Expand all object

ChatRequest > Expand all object

ChatResponse > Expand all object

ConversationDeleteResponse > Expand all object

FareNamesRequest > Expand all object

FareNamesResponse > Expand all object

HTTPValidationError > Expand all object

JourneySegment > Expand all object

SummaryRequest > Expand all object

SummaryResponse > Expand all object

ValidationError > Expand all object

Figure 9 – Swagger UI at /docs

FareLens 0.8.0 GA1.5.1
OpenAPI spec

FareLens turns raw airline fare rules into traveller-friendly summaries, answers questions about them, and cleans up fare-family amenity lists and fare-family names, using the LLM provider you configure (OpenAI, Azure OpenAI, Anthropic, Google Gemini, Groq or AWS Bedrock).
 Authenticate public endpoints with an API key created in the admin UI:
`X-API-Key: FK_...`

health

- GET** `/health` Liveness probe
- GET** `/health/ready` Readiness probe

fare-rules

POST `/api/v1/fare-rules/summary` Summarise fare rules into traveller-friendly Markdown

Turns raw airline fare rules text into a concise Markdown summary covering cancellation/refund, change/issue and no-show conditions. Results are cached (Redis → Postgres) by content digest; the [X-Cache](#) response header reports `valid=ttl`, `postgress=ttl` or `generated`.

Parameters
 No parameters Try it out

Request body required application/json

Example Value Schema

```
{
  "fare_rules_text": "string",
  "lang": "en",
  "is_valid": false,
  "system_error": false
}
```

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json Content-accept header Example Value Schema <pre>{ "summary_markdown": "string", "cancellation": "string", "change": "string", "no_show": "string", "issue": "string", "lang": "string", "is_valid": true, "system_error": false, "latency_ms": 0 }</pre>		
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": { "type": "string", "msg": "string", "type": "string", "param": "string", "ctx": {} } }</pre>		

- POST** `/api/v1/fare-rules/chat/stream` Ask a question about the fare rules (Server-Sent Events)
- POST** `/api/v1/fare-rules/chat` Ask a question about the fare rules (single JSON response)
- DELETE** `/api/v1/fare-rules/chat/{conv_id}` Forget a conversation's stored segments

amenities

- POST** `/api/v1/amenities/summary` De-duplicate, classify and translate fare-family amenities
- POST** `/api/v1/amenities/fare-names` Convert fare-family codes to friendly labels

Schemas

- AmenitiesResponse** Expand all object
- AmenityItem** Expand all object
- ChatMessage** Expand all object
- ChatRequest** Expand all object
- ChatResponse** Expand all object
- ConversationDeleteResponse** Expand all object
- FareNamesRequest** Expand all object
- FareNamesResponse** Expand all object
- HTTPValidationError** Expand all object
- JourneySegment** Expand all object
- SummaryRequest** Expand all object
- SummaryResponse** Expand all object
- ValidationError** Expand all object

Figure 10 – Summary endpoint expanded: request schema and responses

4. Configuration

FareLens works with zero configuration. Copy `.env.example` to `.env` to change anything.

4.1 Environment variables

Variable	Default	Purpose
<code>APP_SECRET_KEY</code>	(generated)	Signs sessions and encrypts stored credentials. Set it in production. If empty, one is generated into <code>APP_DATA_DIR/secret.key</code> .
<code>APP_DATA_DIR</code>	<code>/data</code> (Docker)	Local state: generated secret, datastore overrides.
<code>DATABASE_URL</code> / <code>REDIS_URL</code>	bundled containers	Default datastores.
<code>ADMIN_DEFAULT_USERNAME</code> / <code>ADMIN_DEFAULT_PASSWORD</code>	<code>admin</code> / <code>admin</code>	Seeded once, on first start with an empty database.
<code>MAX_FARE_RULES_CHARS</code>	<code>60000</code>	Max fare-rules length per request / segment.
<code>SUMMARY_CACHE_NAMESPACE</code>	<code>v1</code>	Bump to invalidate all cached summaries (e.g. after editing prompts).
<code>SUMMARY_CACHE_TTL_SECONDS</code>	<code>2592000</code>	Redis TTL for summaries; Postgres keeps them indefinitely.
<code>CONVERSATION_TTL_SECONDS</code>	<code>3600</code>	Retention of chat segments per <code>convo_id</code> .
<code>RATE_LIMIT_REQUESTS</code> / <code>RATE_LIMIT_WINDOW_SECONDS</code>	<code>120</code> / <code>60</code>	Sliding window per API key.
<code>CORS_ALLOW_ORIGINS</code>	<code>*</code>	Comma-separated origins for browser clients.
<code>LLM_TIMEOUT_SECONDS</code>	<code>90</code>	Upstream request timeout.

4.2 LLM providers

Provider	Credentials	Notes
OpenAI	API key, optional base URL	Base URL makes it work with any OpenAI-compatible gateway.
Azure AI (Azure OpenAI)	Endpoint, API key, API version	Model = your deployment name .
Anthropic (Claude)	API key	Sampling parameters are omitted (current Claude models reject them).
Google (Gemini)	API key	Google AI Studio keys.
Groq	API key	Fast open-weight models; Groq retires models regularly.
AWS Bedrock	Region, optional access key/secret	Empty keys → container IAM role. Uses the Converse API, so any Converse-capable model works.

Credentials are Fernet-encrypted with `APP_SECRET_KEY`. The public API never sees them; the admin UI shows them masked and can reveal them to a logged-in admin.

4.3 Using your own Postgres / Redis

The **Data stores** page lets you point FareLens at an existing Postgres and/or Redis with plain host / port / user / password fields (or a pasted connection string). It tests the connection, creates the schema, copies users, LLM configuration and API keys if the target is empty, and switches live — no restart. The choice is persisted, encrypted, in the data volume. Redis is optional: without it, summaries are cached in Postgres only and chat context lives in memory (single instance).

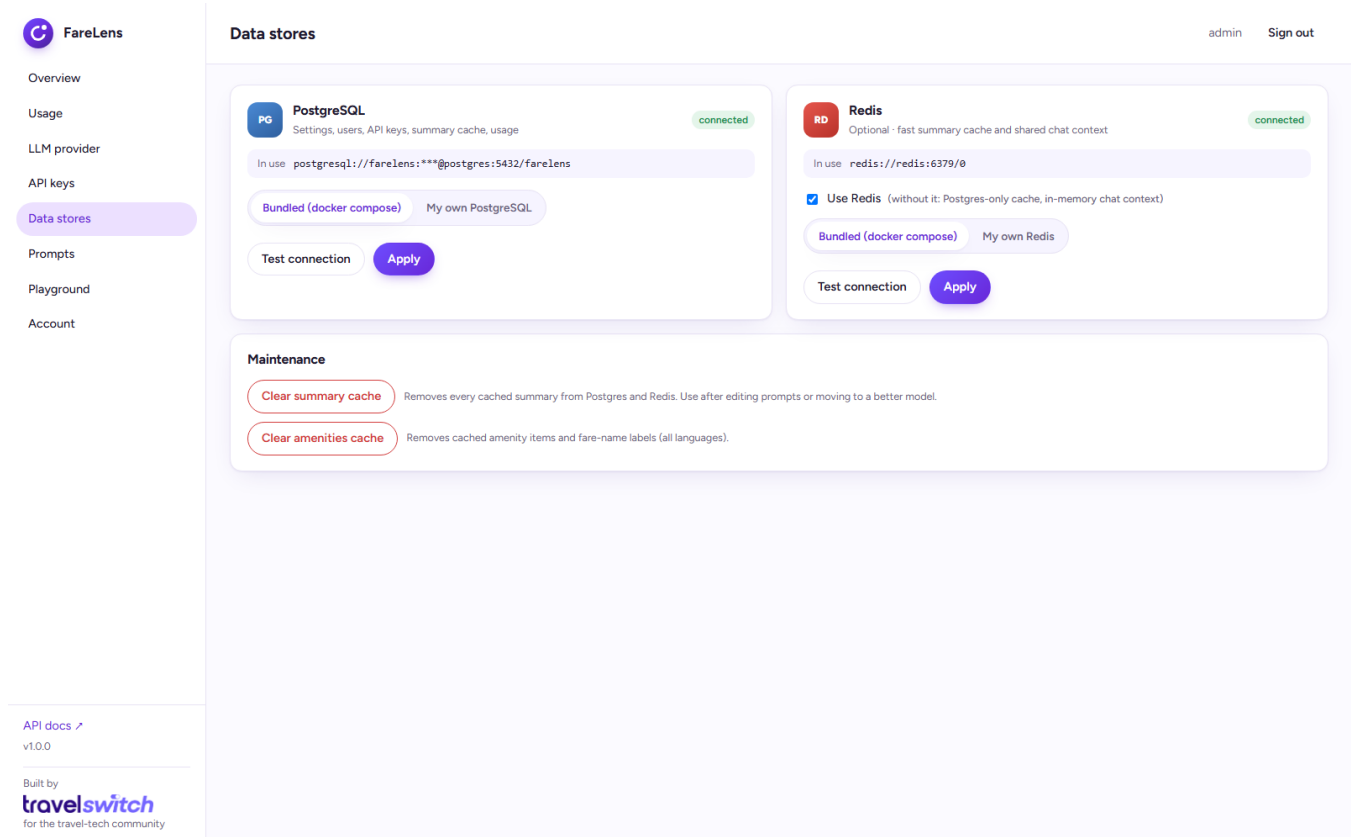


Figure 11 – Data stores: bundled or your own, with connection tests

4.4 Prompts

All prompts, including the amenities and fare-name ones, can be edited from the admin UI (**Prompts** page) or on disk. The UI validates required placeholders, previews the edited prompt against sample fare rules before saving, stores overrides in Postgres (so they apply to every instance and survive upgrades) and offers a one-click reset to the file default. Saving a summary prompt clears the summary cache so the change takes effect immediately.

FareLens

Overview
Usage
LLM provider
API keys
Data stores
Prompts
Playground
Account

Prompts

Templates

SUMMARY

- system prompt default
- user message default
- desktop layout default
- mobile layout default

CHAT

- system prompt default
- desktop layout default
- mobile layout default

AMENITIES

- summary system prompt** default
- summary user message default
- translate system prompt default
- translate user message default

FARE NAMES

- system prompt default
- user message default

Files under prompts/ are the defaults. Edits made here are stored in the database and take effect immediately on every instance.

Amenities · summary system prompt default

Merge duplicates, classify, shorten and translate fare-family benefit texts.

[[TARGET_LANGUAGE]] [[ALLOWED_TYPES]]

You are FareLens, a flight amenities specialist. You turn raw airline "fare family" benefit text (from GDS, NDC or airline feeds) into a short, clean, de-duplicated list a traveller can scan on a booking page.

Output language
Write every 'description' and 'details' value in [[TARGET_LANGUAGE]], using only that language's native script. Keep airport codes, airline codes, currency codes, numbers, units (kg, cm, in) and URLs exactly as they appear.

Input
A JSON array of objects: 'key' (opaque id) and 'text' (raw amenity text). Texts often repeat the same benefit in different words, mix languages, or carry marketing noise.

What to do

1. **Merge duplicates.** If two or more texts mean the same thing (e.g. "1pc x 7kg" and "1 piece, max 7 kg, 56x45x25 cm"), return ONE object and list every matching 'key' in its 'key' array. Every input key must appear in exactly one output object.
2. **Classify** each object with one 'type' from: [[ALLOWED_TYPES]]. Use 'CabinBaggage' for hand/carry-on luggage and 'Baggage' for checked luggage. Use 'Warning' for restrictions that are not benefits (e.g. "no changes permitted"). Use 'Other' only when nothing else fits.
3. **Write a short 'description'**: the main takeaway only, 3 to 10 words, traveller-friendly, no marketing adjectives. Examples: "1 checked bag up to 23 kg", "Lounge access not included", "Seat selection for a fee".
4. **Move qualifiers to 'details'**: dimensions, weight and size limits, timing conditions, airport or route exceptions, fee notes, "subject to availability". Leave 'details' empty only when there is genuinely nothing extra.

3,104 chars Preview language English Mobile layout

Preview with sample Save Show default

API docs v1.0.0

Built by **travelswitch** for the travel-tech community

Figure 12 – Prompt editor with live preview

On disk, prompts live in `prompts/fare_rules/*.md` and are hot-reloaded on change. `summary_system.md` and `chat_system.md` carry the domain rules — ATPCO vocabulary, before/after-departure and no-show logic, a strict no-hallucination policy — while the `*_layout*.md` files control desktop versus mobile formatting. After changing a prompt, bump `SUMMARY_CACHE_NAMESPACE` or clear the cache from the UI.

5. Architecture

5.1 Request flow

1. The client calls the API with an `X-API-Key`; the key hash is verified (cached for 60 s) and the request is rate-limited.
2. Fare-rules text is normalised (HTML stripped, whitespace and case canonicalised) and hashed with SHA-256 together with the language, layout and cache namespace.
3. The digest is looked up in Redis, then Postgres. A hit returns immediately (`X-Cache: redis-hit / postgres-hit`).
4. On a miss, the prompt is rendered from the templates and sent to the configured provider adapter.
5. The result is stored in Postgres and Redis, a usage event is recorded (tokens, latency, cache status, key), and the response is returned.

Chat follows the same provider path but streams tokens over Server-Sent Events; segment context is stored per conversation in Redis (or memory) with a TTL.

5.2 Code layout

```
farelens/
  api/          FastAPI routes (public v1 + admin), auth dependencies, middleware
  services/     summary (cache → LLM), chat (SSE), usage, API keys, users, LLM config
  llm/          provider registry + adapters (openai/azure/groq, anthropic, google, bedrock)
  db/           asyncpg / redis managers, live datastore switching, SQL migrations
  core/         settings, security (bcrypt · JWT · Fernet), error envelope
  prompts/      editable prompt templates
  migrations/   forward-only SQL applied at startup
  ui/           admin SPA (vanilla JS, no build step)
docker-compose.yml · Dockerfile · tests/ · .github/workflows/ci.yml
```

5.3 Technology

Layer	Choice
Runtime	Python 3.12, FastAPI, Uvicorn (async end-to-end)
Datastores	PostgreSQL 16 (asyncpg), Redis 7 (optional)
LLM SDKs	openai (OpenAI, Azure, Groq), anthropic, google-genai, boto3 (Bedrock Converse)
Security	bcrypt passwords, HMAC-signed session cookies with CSRF guard, Fernet-encrypted secrets, SHA-256 API-key hashes
Packaging	Multi-stage Docker image, non-root user, healthchecks; GitHub Actions CI (ruff, pytest, image build)

6. Deployment

6.1 Production checklist

- ☐ Set `APP_SECRET_KEY` to a long random value and never change it afterwards (it encrypts stored credentials).
- ☐ Change the admin password (Account → Change password).
- ☐ Change the bundled Postgres password, or use your own Postgres.
- ☐ Serve over HTTPS behind a reverse proxy; the session cookie becomes `Secure` automatically.
- ☐ Restrict `CORS_ALLOW_ORIGINS` if browsers call the API directly.
- ☐ Keep Postgres and Redis on the private network — the compose file does not publish their ports.

6.2 Reverse proxy

FareLens listens on plain HTTP (port 8000) and honours `X-Forwarded-*` headers. Server-Sent Events require response buffering to be **off** on every proxy in the path.

```
# nginx
location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;          # required for SSE
    proxy_read_timeout 300s;
}
```

6.3 Scaling, persistence and upgrades

- Run several `app` replicas behind the proxy with Redis enabled; provider-config changes propagate within 30 s. Without Redis, keep a single replica.
- Back up the Postgres volume (`pg_dump`) and the `/data` volume (it holds the generated secret when `APP_SECRET_KEY` is not set).
- Upgrade with `git pull && docker compose build app && docker compose up -d`; schema migrations run automatically and are safe to run from multiple replicas.
- Usage rows are small but unbounded — prune `usage_events` periodically if traffic is heavy.

6.4 What FareLens stores

Data	Where	Protection
LLM API keys / cloud credentials	Postgres <code>llm_config.secrets</code>	Fernet-encrypted with <code>APP_SECRET_KEY</code>
External datastore DSNs	<code>APP_DATA_DIR/datastores.json</code>	Fernet-encrypted, file mode 600
Admin passwords	Postgres <code>users</code>	bcrypt (12 rounds)
API keys	Postgres <code>api_keys</code>	SHA-256 hash + display prefix
Fare-rules summaries	Redis + Postgres cache	Plain text (no PII expected)
Chat context (segments)	Redis (TTL) or memory	Plain text, expires
Usage events	Postgres	Metadata only — no request bodies

7. Contributing, credits and license

Issues and pull requests are welcome. Run `ruff check farelens tests` and `pytest -q` before opening a PR; see [CONTRIBUTING.md](#) for the full guide, including how to add a new LLM provider (a registry entry plus an adapter — the UI form is generated automatically).

Security issues should be reported privately as described in [SECURITY.md](#).

Provider icons in the admin UI are from lobe-icons (MIT). Brand names and logos remain the property of their respective owners.

FareLens is released under the MIT License. © 2026 TravelSwitch.

Built by TravelSwitch for the travel-tech community · <https://travelswitch.com>